

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Case Study

COURSE NAME:	Cloud Computing and Big Data Analytics	COURSE CODE:	CSA15
--------------	--	--------------	-------

COURSE OUTCOME COVERED

CO3: Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)

Assessment Tool Weightage: 20%

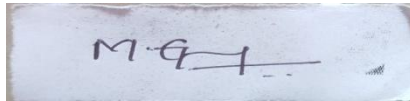
Dave's Taxonomy: Precision (Level 3) Question Count: 3	Total Marks: 30
--	-----------------

Code of Conduct

I Maddirala Gopi Krishna Reddy 192372055 Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Assessment Tasks

Case Study

1. Case 1: A food delivery company wants to deploy a cloud-native backend with web APIs, browser clients, and shared cloud storage for receipts and delivery images. Propose the architecture and explain the role of each component.
2. Case 2: A media startup must decide between SAN-backed storage, object storage, and serverless processing for video upload workflows. Analyze the case and recommend a deployment model.
3. Case 3: A smart city platform needs low-latency decision making near sensors while still using the cloud for analytics. Analyze the use of edge computing and fog computing in this environment.

Case Study Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Case	25%	Clearly identifies all technical and business requirements	Identifies most requirements	Basic understanding only	Poor understanding of case
Application of Concepts	25%	Applies suitable cloud concepts and tools effectively	Good application with minor issues	Partial application	Weak or incorrect application

Analysis	20%	Strong analysis with clear trade-off reasoning	Reasonable analysis	Limited analysis	Minimal analysis
Solution Design	20%	Solution is feasible, practical, and well justified	Reasonable solution with minor gaps	Basic solution	Unclear solution
Clarity	10%	Well-structured and easy to follow	Mostly clear	Somewhat unclear	Poorly organized

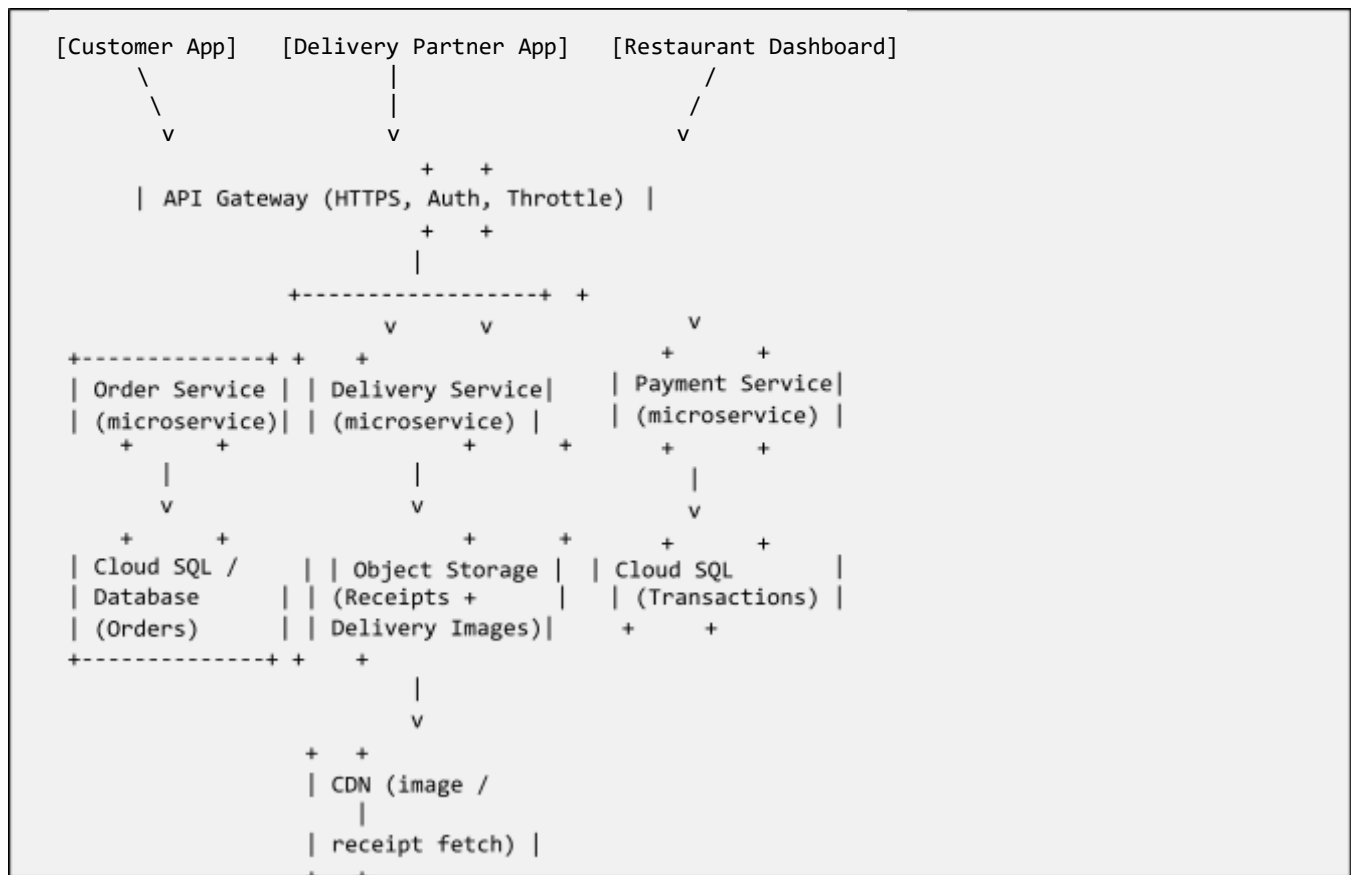
Case 1: Cloud-Native Backend for a Food Delivery Company

1. Case Understanding

The food delivery company needs a cloud-native backend that serves three types of clients — customer mobile/web apps, delivery-partner apps, and restaurant dashboards — all interacting through web APIs. The system must also durably store receipts (PDF/text) and delivery images (proof-of-delivery photos), which are unstructured, high-volume, and infrequently modified once created. The core requirements are therefore: a scalable API layer for real-time order and delivery operations, browser/mobile client support, and durable, cost-effective shared storage for receipts and images that multiple services (billing, support, delivery tracking) can access.

Key business drivers include high request volume during peak meal hours (elastic scaling), low-latency order/status updates, secure access to customer and payment-adjacent data, and long-term retention of receipts and delivery-proof images for dispute resolution and compliance.

2. Proposed Architecture



3. Role of Each Component

Component	Role
API Gateway	Single entry point for all clients; handles HTTPS termination, authentication, rate limiting, and routing requests to the correct microservice

Browser / Mobile Clients	Present order placement, delivery tracking, and dashboard UIs; communicate exclusively via REST/HTTPS calls to the API Gateway
Order / Delivery / Payment Microservices	Independently deployable compute units (containers) implementing business logic; scale independently based on load
Object Storage (Receipts & Images)	Durable, highly available store for receipt documents and delivery-proof images; accessed via signed URLs from the services
Relational Database	Stores structured, transactional data — orders, delivery status, payment records — requiring ACID guarantees
CDN	Caches and serves receipt/image content close to the user, reducing latency and offloading repeated reads from object storage
Identity/Auth Service	Issues and validates tokens for customers, delivery partners, and restaurant staff with role-based access control

4. Recommended Deployment Model

A microservices architecture deployed on managed containers (e.g., ECS/EKS, AKS, or GKE) behind an API Gateway is recommended over a single monolithic backend, because order volume, delivery tracking, and payment processing have different scaling profiles and failure isolation needs. Each microservice can be auto-scaled independently — for example, the delivery-tracking service may need to scale sharply during lunch/dinner peaks while the payment service scales more steadily.

Receipts and delivery images should be stored in object storage rather than a database or block volume, since they are immutable, unstructured files accessed occasionally for support/dispute purposes; object storage offers near-unlimited scalability and low per-GB cost with lifecycle policies (e.g., moving older receipts to a cheaper archive tier after 90 days). A CDN in front of object storage further reduces latency and cost for repeated reads such as a customer re-viewing a past receipt.

5. Conclusion

The proposed cloud-native architecture separates concerns cleanly: an API Gateway and microservices handle dynamic, transactional workloads, while object storage and a CDN handle static, high-volume unstructured content such as receipts and delivery images. This separation allows each layer to be scaled, secured, and billed independently, aligning infrastructure cost with actual usage patterns.

For the food delivery company, this design ensures the system can absorb sudden spikes in order volume without impacting the durability or retrieval speed of historical receipts and images, while also providing a clear path to add new client types (e.g., a support-agent portal) without re-architecting the core backend.

Case 2: Storage and Processing Strategy for a Media Startup's Video Upload Workflow

1. Case Understanding

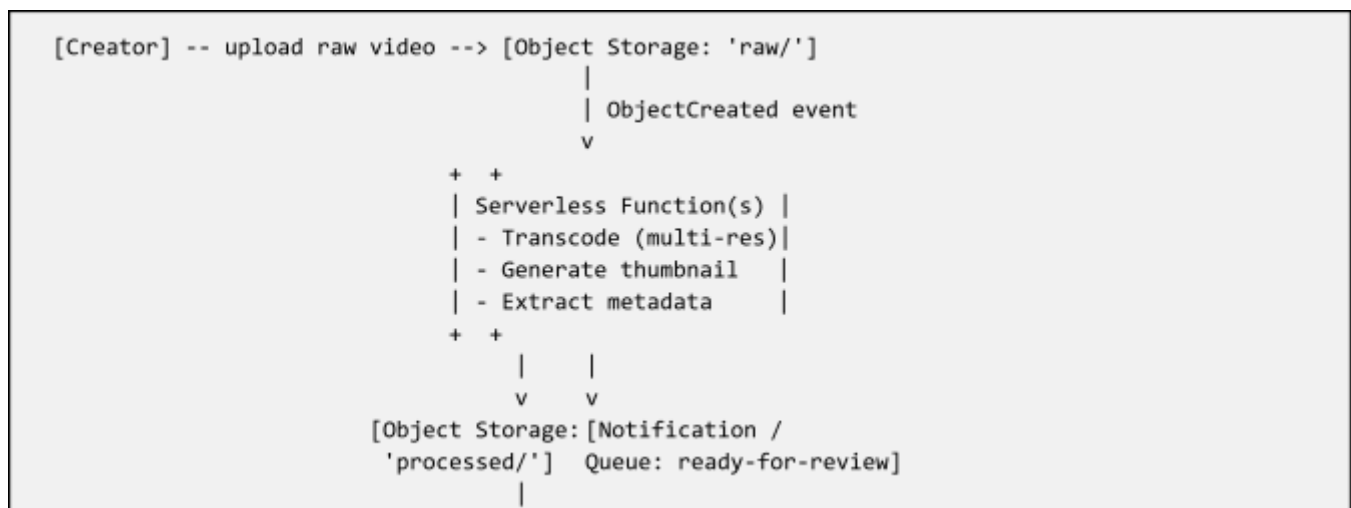
The media startup runs a video-upload workflow where creators upload raw video files that must be stored, transcoded into multiple resolutions/formats, and eventually served to viewers. The company must choose among three storage/processing strategies: SAN-backed (block) storage, object storage, and serverless processing. The decision hinges on expected data volume (likely large and growing), access pattern (write-once, read-many, with a processing step in between), scalability needs, and the startup's cost sensitivity as a young company that cannot over-provision infrastructure.

The workflow has two distinct phases: an unpredictable, spiky ingestion phase (creators uploading at any time) and a required transformation phase (transcoding, thumbnail generation) before content is servable to end users. Each phase has different infrastructure implications that must be analyzed separately.

2. Analysis of Options

Option	Strengths	Weaknesses for this Case
SAN-backed (Block) Storage	Very low latency, high IOPS, good for databases/VM disks	Fixed capacity per volume, expensive at scale, not natively accessible via HTTP/API, requires attached compute — poor fit for large, growing, infrequently-modified video files
Object Storage	Virtually unlimited scale, low cost per GB, native REST API access, built-in event triggers, versioning and lifecycle policies	Higher per-request latency than block storage — but irrelevant for write-once/read-many video files
Serverless Processing (FaaS)	Automatically scales with upload volume, pay-per-execution (no idle transcoding servers), integrates natively with object storage events	Execution time limits may require chunking very long videos; cold-start latency for infrequent triggers

3. Recommended Deployment Model



v
[CDN] --> [Viewers]

Object storage is recommended for both raw and processed video files because the access pattern is write-once/read-many at a scale that will grow unpredictably as the startup gains creators — a profile object storage is specifically designed for, at a fraction of SAN's cost per GB. SAN-backed block storage is not recommended as the primary media store; it is best reserved for the startup's transactional database (e.g., user accounts, video metadata index) where low-latency random access is genuinely required.

Serverless (FaaS) processing is recommended for the transcoding/thumbnail pipeline because upload volume is inherently spiky (bursts when popular creators post) and unpredictable for an early-stage startup; a serverless model scales functions automatically per upload event and avoids paying for idle transcoding servers during quiet periods, which directly matches the startup's cost-sensitivity.

4. Trade-off Reasoning

The trade-off is primarily between control/latency (SAN) and elasticity/cost-efficiency (object storage + serverless). A startup with unpredictable growth and limited capital should prioritize elasticity and pay-per-use pricing over the marginal latency benefits of SAN, since video content does not require sub-millisecond access — a few hundred milliseconds for object retrieval is imperceptible for on-demand video playback, especially once a CDN is layered in front.

A secondary trade-off is serverless execution limits versus dedicated transcoding servers: for very large or long-form video files, the startup may need to chunk transcoding jobs or fall back to a managed container-based transcoding service for outliers, while keeping serverless as the default path for the majority of shorter uploads.

5. Conclusion

For the media startup's video upload workflow, the recommended deployment model is object storage for both raw and processed media combined with serverless functions for transcoding and thumbnail generation, fronted by a CDN for delivery to viewers. SAN-backed block storage should be reserved only for the startup's relational metadata store, not the media files themselves.

This combination gives the startup near-unlimited storage scalability, cost that scales directly with actual usage (both storage and compute), and an architecture that can absorb unpredictable creator-driven traffic spikes without requiring the startup to provision or manage dedicated transcoding infrastructure in advance.

Case 3: Edge and Fog Computing for a Smart City Platform

1. Case Understanding

The smart city platform collects data from a large number of distributed sensors (traffic cameras, air-quality sensors, streetlights, public-safety devices) spread across the city. Certain decisions — such as adjusting a traffic signal or triggering a local alarm — must happen with minimal delay at or near the sensor, while city-wide trends, historical reporting, and cross-department analytics require aggregating data centrally in the cloud. The case therefore requires an architecture that balances low-latency local autonomy with centralized, large-scale analytics.

The core challenge is that sending every raw sensor reading directly to a distant cloud data center for every decision would introduce network latency, consume significant bandwidth, and create a single point of failure for time-critical local actions — making a purely cloud-centric design unsuitable for this use case.

2. Analysis: Edge and Fog in the Smart City Context

Tier 1 - EDGE (at the sensor / device) [Traffic Camera]-> local inference -> immediate action [Air-Quality Sensor]-> threshold check -> local alert Tier 2 - FOG (neighbourhood / district gateway) [Cluster of sensors in a district] -> Fog Node - Aggregates & correlates data across nearby devices - Coordinates district-level responses - Filters/compresses data before forwarding Tier 3 - CLOUD (city data center) [City-wide Analytics Platform] - Long-term storage, historical trend analysis - Machine-learning model training - Cross-department dashboards & reporting - Redeploys updated models down to edge/fog			
Tier	Latency	Scope of Decision	Example in Smart City
Edge	Milliseconds	Single device/sensor	Traffic camera adjusting a single signal instantly
Fog	Tens of milliseconds	Cluster/district of devices	Coordinating signal timing across a street or synchronising streetlights in a block
Cloud	Seconds and beyond (batch)	City-wide	Historical congestion analysis,

			retraining prediction models, cross-agency dashboards
--	--	--	---

3. Recommended Use in this Environment

Edge computing should be used for any decision that must happen within milliseconds and does not require context beyond the immediate sensor — for example, an air-quality sensor triggering a local alert when a hazardous threshold is crossed, or a single traffic camera adjusting its own signal based on immediate vehicle count, without waiting on any network round trip.

Fog computing should be introduced at a district/neighbourhood gateway level to aggregate and correlate data from clusters of nearby edge devices — for instance, coordinating signal timing across several intersections on the same corridor, or correlating multiple air-quality sensors in a district to detect a spreading pollution event that no single sensor could identify alone. The fog layer also reduces the volume of raw data forwarded to the cloud by filtering and summarising it locally.

The cloud layer should be reserved for functions that are not time-critical and benefit from city-wide scale: long-term historical storage, training and refining machine-learning models on aggregated data, and providing dashboards for city planners and multiple departments. Updated models trained in the cloud can then be redeployed down to the edge and fog tiers, closing the loop.

4. Trade-off Reasoning

The central trade-off is responsiveness versus contextual scope: edge computing offers the lowest latency but only local context, fog computing offers moderate latency with cluster-level context, and cloud computing offers the richest, city-wide context but with the highest latency and dependency on network connectivity. A smart city platform that relied solely on the cloud would fail time-critical local actions during network congestion or outages, whereas a platform that relied solely on the edge would be unable to detect patterns that only emerge across multiple sensors or over long time horizons.

5. Conclusion

For the smart city platform, a three-tier edge-fog-cloud architecture is the most suitable design. Edge nodes provide the millisecond-level autonomy required for immediate local actions and safety-critical responses; fog nodes provide district-level coordination and data reduction between edge devices and the cloud; and the cloud provides the city-wide analytics, long-term storage, and model-training capability that neither edge nor fog can support alone.

This layered approach ensures the platform remains responsive and resilient at the local level even during network disruption, while still delivering the centralized intelligence and historical insight that city administrators need to plan and optimize services across the entire city.